# Project Proposal: Steam Game Recommendation System

**Môn học:** IT4868 – Web Mining (SoICT, HUST)
**Giảng viên:** TS. Dương Quang Huy
**Chủ đề:** Recommendation Systems
**Hướng tiếp cận:** Academic (định nghĩa bài toán, phương pháp đề xuất, thực nghiệm trên benchmark + code, kết luận)

**Thành viên nhóm:**

| STT | Họ và tên | MSSV |
|---|---|---|
| 1 | Trần Quang Huy | 202416706 |
| 2 | Phạm Châu Long | 202416714 |
| 3 | Vũ Hồng Nam | 202416728 |
| 4 | Phạm Minh Hiếu | 202416693 |


## 1. Bài toán (Problem Definition)

### 1.1. Bối cảnh

Steam là nền tảng phân phối game PC lớn nhất thế giới, với hơn 100.000 tựa game và hàng chục triệu người dùng hoạt động. Số lượng game mới phát hành mỗi năm rất lớn khiến người dùng khó tự tìm được game phù hợp. Hệ thống gợi ý (recommender system) là thành phần cốt lõi giúp cá nhân hoá trải nghiệm và tăng khả năng khám phá game, đặc biệt với các game indie ít người biết.

Khác với dữ liệu phim (MovieLens) có rating 1–5 sao, dữ liệu Steam chủ yếu là **implicit feedback**: người dùng sở hữu game, thời gian chơi (playtime), và review dạng nhị phân (Recommended / Not Recommended). Điều này đặt ra các thách thức riêng về mô hình hoá.

### 1.2. Phát biểu bài toán

Bài toán **Top-K Recommendation từ implicit feedback**:

- **Input:**
  - Tập người dùng $U$, tập game $I$.
  - Lịch sử tương tác của mỗi user $u$: chuỗi các game $(i_1, i_2, \ldots, i_n)$ kèm thời điểm, playtime và nhãn review (recommend hay không).
  - Metadata của game: tags, thể loại, mô tả, giá, ngày phát hành.
- **Output:** Danh sách xếp hạng $K$ game (ví dụ $K = 10, 20$) mà user $u$ chưa tương tác và có khả năng cao sẽ chơi/thích tiếp theo.

Về mặt hình thức: học hàm điểm $\hat{y}_{ui} = f(u, i \mid \Theta)$ và trả về $\text{Top-}K_{i \in I \setminus I_u}\ \hat{y}_{ui}$.

### 1.3. Thách thức (Challenges)

1. **Implicit feedback & thiếu negative:** Game không tương tác chưa chắc là "không thích" — có thể chỉ là chưa biết đến.
2. **Dữ liệu cực kỳ thưa (sparsity):** Mỗi user chỉ tương tác với một phần rất nhỏ trong hàng chục nghìn game (density thường < 0.1%).
3. **Popularity bias / long-tail:** Một số ít game AAA chiếm phần lớn tương tác; mô hình dễ chỉ gợi ý game phổ biến.
4. **Cold-start:** Game mới phát hành có rất ít hoặc không có tương tác.
5. **Tính tuần tự (sequential):** Sở thích người dùng thay đổi theo thời gian; game chơi gần đây phản ánh sở thích hiện tại tốt hơn.
6. **Quy mô dữ liệu lớn:** Hàng chục triệu tương tác — cần xử lý hiệu quả trong giới hạn tài nguyên (Google Colab).

### 1.4. Câu hỏi nghiên cứu (Research Questions)

- **RQ1:** Các mô hình deep learning dạng graph (LightGCN) và sequential (SASRec) cải thiện bao nhiêu so với các baseline CF cổ điển trên dữ liệu Steam?
- **RQ2:** Việc đưa thông tin nội dung game (tags, mô tả) vào item embedding có cải thiện gợi ý cho game cold-start và long-tail không?
- **RQ3:** Gán trọng số tương tác theo playtime và nhãn review có cho thứ hạng tốt hơn so với coi mọi tương tác là như nhau không?

## 2. Phương pháp (Proposed Methods)

Nhóm sẽ triển khai và so sánh 3 nhóm phương pháp, từ cổ điển đến Deep Learning, và đề xuất một cải tiến hybrid.

### 2.1. Baselines

| Mô hình | Mô tả ngắn | Lý do chọn |
|---|---|---|
| **MostPop** | Gợi ý các game phổ biến nhất mà user chưa có | Baseline tối thiểu, đo mức độ popularity bias |
| **ItemKNN** | Collaborative filtering dựa trên cosine similarity giữa các item | Phương pháp CF cổ điển, dễ giải thích |
| **BPR-MF** | Matrix Factorization tối ưu Bayesian Personalized Ranking loss | Chuẩn mực cho implicit feedback |

### 2.2. Mô hình chính (Deep Learning)

**(a) LightGCN** (He et al., SIGIR 2020) — *Graph-based CF*

- **Biểu diễn input:** Đồ thị hai phía user–game; mỗi node có embedding khởi tạo $e^{(0)}$.

- **Lan truyền:** $e_u^{(k+1)} = \sum_{i \in N_u} \frac{1}{\sqrt{|N_u||N_i|}} e_i^{(k)}$ (tương tự cho item), bỏ feature transformation và non-linearity.
- **Embedding cuối:** trung bình có trọng số các layer; điểm $\hat{y}_{ui} = e_u^\top e_i$.
- **Huấn luyện:** BPR loss với negative sampling.
- **Động lực:** Khai thác high-order connectivity trên đồ thị (liên hệ trực tiếp với phần *Link Analysis* của môn học), giảm ảnh hưởng của sparsity.

**(b) SASRec** (Kang & McAuley, ICDM 2018) — *Sequential recommendation với Transformer*

- **Biểu diễn input:** Chuỗi $n$ game gần nhất của user (theo thời gian), item embedding + positional embedding.
- **Mô hình:** Các khối self-attention một chiều (causal) để dự đoán game kế tiếp.
- **Huấn luyện:** Binary cross-entropy với 1 negative sample mỗi vị trí.
- **Dự đoán:** Lấy hidden state cuối cùng, tính dot-product với toàn bộ item embedding.
- **Động lực:** Nắm bắt sự thay đổi sở thích theo thời gian; SASRec được đánh giá gốc trên chính dataset Steam nên có kết quả tham khảo.

### 2.3. Đề xuất cải tiến: Content-aware Hybrid

Để xử lý **cold-start** và **long-tail**, nhóm đề xuất kết hợp thông tin nội dung game vào mô hình CF:

1. **Text encoding:** Dùng Sentence-BERT (`all-MiniLM-L6-v2`) mã hoá chuỗi `title + tags + description` của mỗi game thành vector $c_i \in \mathbb{R}^{384}$.
2. **Fusion:** Item embedding khởi tạo trong LightGCN/SASRec được tính bằng $e_i^{(0)} = W c_i + \text{id_emb}_i$, trong đó $W$ là ma trận chiếu học được. Với game mới (chưa có ID embedding đã học), mô hình vẫn có thể dùng $W c_i$.
3. **Trọng số theo hành vi (tuỳ chọn):** Dùng playtime và nhãn review (Opinion Mining ở mức đơn giản) để gán confidence cho mỗi tương tác, ví dụ $w_{ui} = 1 + \alpha \log(1 + \text{playtime}_{ui})$, và giảm trọng số các tương tác có review "Not Recommended".

**Giả thuyết kiểm chứng:** Mô hình hybrid cải thiện đáng kể Recall/NDCG trên tập game ít tương tác (cold/long-tail items) mà không làm giảm hiệu năng tổng thể.

## 3. Dữ liệu (Benchmark Datasets)

Cả hai bộ dữ liệu đều công khai, được sử dụng rộng rãi trong các nghiên cứu về recommender system:

| Dataset | Nguồn | Quy mô | Nội dung |
|---|---|---|---|
| **Steam Video Game and Bundle Data** (McAuley Lab, UCSD) | https://cseweb.ucsd.edu/~jmcauley/datasets.html#steam_data | ~7.8M review; ~88K user, ~10K game, ~5M lượt sở hữu (Australian users) | Review (có ngày đăng), danh sách game sở hữu + playtime (không có timestamp), metadata game. Phần review là benchmark dùng trong paper SASRec. |
| **Game Recommendations on Steam** (Kaggle, A. Kozyriev, 2023) | https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam | ~41M review, ~14M user, ~50K game | `recommendations.csv` (user, game, is_recommended, hours, date), `games.csv`, `games_metadata.json` (description, tags) |

**Tiền xử lý:**

- Lọc **k-core** (giữ user và game có ≥ 5 tương tác) để giảm nhiễu và kích thước.
- Với dataset Kaggle, lấy mẫu con (ví dụ ~1M user) để vừa tài nguyên Colab.
- **Định nghĩa tương tác:** Với dataset UCSD, dữ liệu sở hữu game không có timestamp nên mỗi **review** (có ngày đăng) được dùng làm một tương tác, giống cách làm của paper SASRec. Playtime từ bảng sở hữu game được ghép theo (user, game) và chỉ dùng làm trọng số tương tác. Dataset Kaggle có sẵn `date`, `hours`, `is_recommended` cho mỗi tương tác.
- Sắp xếp tương tác theo thời gian cho mô hình sequential.
- **Chia dữ liệu:** Leave-one-out theo thời gian (tương tác cuối → test, áp chót → validation, còn lại → train), thống nhất cho tất cả mô hình.
- **Tập cold-start:** Tách riêng ~10% game ra khỏi tập train để đánh giá khả năng gợi ý game mới.

## 4. Triển khai (Implementation)

- **Thư viện:**
  - Xử lý dữ liệu: `pandas`, `numpy`, `scipy.sparse`
  - Mô hình: `PyTorch`, `RecBole` (framework có sẵn BPR, ItemKNN, LightGCN, SASRec — dùng làm nền tảng và để đảm bảo tính tái lập)
  - Content features: `sentence-transformers`
  - Trực quan hoá: `matplotlib`, `seaborn`
- **Phần nhóm tự cài đặt/chỉnh sửa (so với code có sẵn):**
  1. Pipeline tiền xử lý hai dataset Steam sang định dạng atomic file của RecBole.
  2. Module content-aware item embedding (Sentence-BERT + projection) tích hợp vào LightGCN và SASRec.
  3. Cơ chế trọng số tương tác theo playtime/review trong loss function.
  4. Script đánh giá riêng cho cold-start và long-tail, cùng các metric beyond-accuracy.
- **Tính tái lập:** Mọi thực nghiệm dùng random seed cố định và file cấu hình chung; code và notebook được công khai trên GitHub để có thể chạy lại trên Colab.

## 5. Đánh giá (Evaluation)

### 5.1. Metrics

| Metric | Ý nghĩa | Lý do sử dụng |
|---|---|---|
| **Recall@K** (K = 10, 20) | Tỉ lệ game đúng nằm trong top-K | Đo khả năng "bắt trúng" game user thực sự chơi |
| **NDCG@K** | Có tính đến vị trí của game đúng trong danh sách | Đánh giá chất lượng thứ hạng — gợi ý đúng ở vị trí cao quan trọng hơn |
| **MRR** | Nghịch đảo thứ hạng của game đúng đầu tiên | Bổ sung góc nhìn ranking |
| **Catalog Coverage** | Tỉ lệ game xuất hiện trong ít nhất một danh sách gợi ý | Đo mức độ đa dạng, phát hiện popularity bias |
| **Novelty / Long-tail ratio** | Tỉ lệ game ít phổ biến trong gợi ý | Đánh giá khả năng giúp khám phá game indie |

- Sử dụng **full ranking** trên toàn bộ item (không dùng sampled metrics) theo khuyến nghị của Krichene & Rendle (KDD 2020) để kết quả đáng tin cậy.
- Báo cáo riêng kết quả trên 3 nhóm: toàn bộ, long-tail items, cold-start items.

### 5.2. Thiết kế thực nghiệm

1. So sánh tất cả mô hình (MostPop, ItemKNN, BPR-MF, LightGCN, SASRec, Hybrid) trên cả hai dataset.
2. **Ablation study:** hybrid có/không có content features; có/không có trọng số playtime.
3. **Phân tích siêu tham số:** số layer của LightGCN, độ dài chuỗi và số attention head của SASRec, embedding dimension.
4. **Phân tích định tính:** Case study một vài user — so sánh gợi ý giữa các mô hình.
5. Thảo luận điểm mạnh / điểm yếu của từng phương pháp (độ chính xác, thời gian huấn luyện, khả năng giải thích, xử lý cold-start).

### 5.3. Kết quả kỳ vọng

- Các mô hình Deep Learning (LightGCN, SASRec) vượt trội baseline về Recall/NDCG.
- SASRec có lợi thế trên dataset có thông tin thời gian rõ ràng.
- Mô hình hybrid cải thiện rõ rệt trên cold-start và tăng coverage.

## 6. Kế hoạch thực hiện (Timeline)

| Công việc | Sản phẩm |
|---|---|
| Chốt đề tài, nộp proposal | `proposal.md` |
| Thu thập, khám phá dữ liệu (EDA), tiền xử lý | Notebook EDA, dữ liệu sạch |
| Cài đặt và chạy baselines (MostPop, ItemKNN, BPR-MF) | Kết quả baseline |
| Huấn luyện LightGCN, SASRec | Kết quả mô hình chính |
| Cài đặt module hybrid content-aware, trọng số playtime | Mô hình đề xuất |
| Ablation study, phân tích siêu tham số, thảo luận kết quả | Bảng kết quả, biểu đồ |
| Hoàn thiện báo cáo LaTeX (Overleaf), slide, thuyết trình | Báo cáo (≤ 30 trang), slide, code GitHub |

## 7. Rủi ro và phương án dự phòng

| Rủi ro | Phương án |
|---|---|
| Dữ liệu quá lớn cho Colab | Lọc k-core chặt hơn, lấy mẫu con user; dùng sparse matrix |
| Thời gian huấn luyện GPU bị giới hạn | Lưu checkpoint, giảm embedding dim, ưu tiên dataset UCSD (nhỏ hơn) |
| Metadata thiếu cho một số game | Dùng tags/genre thay mô tả; game thiếu hoàn toàn thì chỉ dùng ID embedding |
| Hybrid không cải thiện | Vẫn báo cáo kết quả ablation và phân tích nguyên nhân như một kết luận có giá trị |